High Performance Programming Project Report

2D Game of Life in C with Serial Optimization and Pthreads Parallelization

Sotirios Oikonomou

March 19, 2026

Abstract

This project implements Conway's Game of Life in C and studies both serial optimization and shared-memory parallelization with Pthreads. The goal was to produce an efficient implementation that is still simple enough to explain clearly. The final program uses a flat one-dimensional `uint8_t` grid to represent a two-dimensional board, double buffering with pointer swapping, and a Pthreads implementation based on persistent worker threads. Since `pthread_barrier_t` is not portable on macOS, the synchronization is implemented using one mutex and condition variables. The results show that serial constant-factor optimizations already improve performance by roughly 4.5×–5.6×, and that the parallel version scales well up to around 8 threads on the tested machine before memory bandwidth and synchronization overhead begin to limit further gains.

1 Introduction

Conway's Game of Life is a classical cellular automaton defined on a two-dimensional grid [1, 2]. Each cell is either alive or dead, and its next state depends on the number of live neighbors in the surrounding 3×3 neighborhood. Although the update rule is simple, the computation is applied repeatedly to every cell in the grid, making the problem a useful benchmark for studying performance optimization and parallelization.

I chose Game of Life because it is small enough to implement cleanly in C, but rich enough to demonstrate important high-performance programming concepts: memory layout, cache behavior, hot-loop optimization, decomposition of work across threads, and synchronization between timesteps. The algorithm is also a good example of a stencil-like computation, where each output element depends on a fixed local neighborhood [8, 9].

2 Problem Description

The program simulates the Game of Life on a rectangular grid of width W and height H for a given number of timesteps S. A dead cell becomes alive if it has exactly three live neighbors. A live cell survives if it has two or three live neighbors; otherwise it dies.

The boundary condition used in this project is a fixed-dead border. Therefore only the interior cells are updated, while the border cells are kept at zero.

The asymptotic time complexity per timestep is

$$T(W, H) = O(W \cdot H),$$

since each cell is processed once. The main performance question is therefore not how to reduce asymptotic complexity, but how to reduce the constant factor and how well the work can be distributed across multiple cores.

3 Implementation Overview

3.1 Data Representation

The problem is two-dimensional, but the grid is stored internally as a flat one-dimensional array:

`grid[y * W + x]`.

This choice keeps memory contiguous, reduces pointer indirection, and is more cache-friendly than a pointer-based 2D array [5, 6]. It also makes row-based partitioning across threads straightforward.

Each cell is stored as `uint8_t`. Since a cell only needs two states, using one byte per cell reduces memory footprint and memory bandwidth pressure compared with larger integer types.

3.2 Double Buffering

The implementation uses two grids:

- `grid`: current generation
- `next`: next generation

At each timestep, the next state is written into `next`, and then the two pointers are swapped:

```
uint8_t *tmp = grid;
grid = next;
next = tmp;
```

This avoids in-place update hazards and avoids copying the whole array after each timestep.

3.3 Serial Baseline

The serial code updates only the interior cells and keeps the borders dead. The implementation was first made correct and then optimized so that the serial version could serve as a meaningful baseline for the later parallel version.

4 Serial Optimizations

The original serial version was correct but contained avoidable overhead in the hot loop. The main optimizations were:

1. **Updating only interior cells.** This removes repeated boundary checks from the hot loop.
2. **Inlining the neighbor sum.** Instead of a generic `count_neighbors()` routine with nested loops, the eight neighbor accesses are summed directly.
3. **Reducing repeated index arithmetic.** Row offsets such as `(i-1) * W`, `i * W`, and `(i+1) * W` are computed once per row and reused.
4. **Using a flat contiguous memory layout.** This improves spatial locality and keeps memory accesses simple.
5. **Using double buffering with pointer swapping.** This avoids an additional full-grid copy after each timestep.

Both the original and optimized serial implementations still have complexity $O(W \cdot H)$ per timestep, but the optimized version reduces the constant factor substantially, which is often crucial in performance-critical loop nests [7, 9].

4.1 Serial Performance Before and After Optimization

Table 1 summarizes the serial optimization results from the earlier development stage.

Table 1: Serial performance before and after optimization (100 steps).

Grid size	Version	Best time (s)	Mean time (s)	Best updates/s
500×500	Original	0.033024	0.033646	7.57×10^8
	Optimized	0.005861	0.006787	4.27×10^9
1000×1000	Original	0.122238	0.128496	8.18×10^8
	Optimized	0.026820	0.027823	3.73×10^9
2000×2000	Original	0.472692	0.502454	8.46×10^8
	Optimized	0.088200	0.099076	4.54×10^9

The resulting best-case serial speedups were:

$$500\times500:\ 5.63\times, \qquad 1000\times1000:\ 4.56\times, \qquad 2000\times2000:\ 5.36\times .$$

These results show that the largest gains came from removing unnecessary work inside the innermost loop. The optimized implementation does not change the algorithm, but it executes the algorithm with much less overhead.

5 Parallelization with Pthreads

5.1 Work Decomposition

The interior rows of the grid are split into almost equal contiguous row blocks. Each thread is responsible for one block. This decomposition is simple, cache-friendly, and easy to explain.

During one timestep, all threads read from the same current buffer and write to disjoint rows of the next buffer. Therefore there are no write-write conflicts in the main computation.

5.2 Why Persistent Threads

A simple design would be to create and join threads at every timestep, but repeated thread management introduces avoidable overhead in iterative parallel programs [3]. Instead, the final version uses persistent worker threads that are created once and reused across all generations.

This design is more efficient because it removes repeated thread creation and join overhead from the timestep loop.

5.3 Synchronization Strategy

The main synchronization requirement is phase synchronization between generations: no worker should start generation $t + 1$ before all workers have finished generation t.

Because `pthread_barrier_t` is not portable on macOS, I implemented synchronization with mutexes and condition variables, which are standard Pthreads synchronization mechanisms [3, 4]:

- one mutex,
- one condition variable to signal that a new step is ready,
- one condition variable to signal that all workers have finished the current step.

The synchronization protocol is:

1. The main thread publishes the current buffers for the next generation.

2. The worker threads wake up and compute their own row blocks.
3. Each worker increments a completion counter.
4. The main thread waits until all workers have completed the current timestep.
5. The buffers are swapped and the next generation begins.

The main thread also computes one row block itself instead of staying idle. This improves utilization and avoids leaving one core unused.

5.4 Why This is Race-Free

The parallel update is race-free because:

- all threads only *read* from the current grid,
- each thread writes to a disjoint range of rows in the next grid,
- buffer swapping happens only after all threads have completed the current generation.

6 Experimental Setup

Experiments were run on:

- Apple M4 Pro (`arm64`)
- 14 CPU cores reported by `sysctl`
- 51,539,607,552 bytes of memory (about 48 GB)
- Apple clang 17.0.0
- macOS Darwin 25.3.0

The code was compiled in C11 mode with optimization and Pthreads support. For correctness/debugging, I also used sanitizer builds with AddressSanitizer and UndefinedBehaviorSanitizer [11, 12].

For each timing configuration, the program was executed five times and both the best and mean runtimes were considered. The performance metric reported by the code is updates per second, computed over the interior cells:

$$\text{updates/sec} = \frac{(W-2)(H-2)S}{\text{elapsed time}}.$$

7 Correctness Verification

Correctness was checked at multiple stages.

First, the serial implementation was validated using canonical Game of Life patterns such as:

- a single live cell, which should die,
- a 2×2 block, which should remain stable,
- a blinker, which should oscillate with period 2.

Second, the final printed grids were compared between the serial path (`-t 1`) and multi-threaded runs using the same seed and parameters. In the final code, performance statistics are printed to `stderr`, so the printed grid on `stdout` can be compared directly with `diff`. This makes output comparison clean and reliable.

Finally, sanitizer builds were used to check for memory errors and undefined behavior during execution. As a final portability and memory-safety check, the program was also compiled and executed on the Uppsala University Linux servers. The serial and parallel outputs matched for the tested cases, and Valgrind reported no memory leaks or memory-access errors.

8 Parallel Performance Results

8.1 Medium Case: 1000×1000, 100 Steps

Table 2 shows the measured results for the medium problem size.

Table 2: Parallel scaling for 1000×1000, 100 steps.

Threads	Best time (s)	Mean time (s)	Mean speedup	Mean efficiency	Best updates/s
1	0.021490	0.025090	1.00	1.000	4.635×10^9
2	0.012862	0.013139	1.91	0.955	7.744×10^9
4	0.007835	0.007902	3.18	0.794	1.271×10^{10}
8	0.006826	0.006890	3.64	0.455	1.459×10^{10}
12	0.007113	0.007392	3.39	0.283	1.400×10^{10}
14	0.007926	0.008391	2.99	0.214	1.257×10^{10}

The medium case scales well up to 4 threads and still improves at 8 threads, but performance begins to flatten and then decline beyond that. This indicates that for this problem size the available parallel work is no longer large enough to amortize all overheads, and memory effects start to dominate.

8.2 Large Case: 2000×2000, 200 Steps

Table 3 shows the corresponding results for a larger and more computationally demanding case.

Table 3: Parallel scaling for 2000×2000, 200 steps.

Threads	Best time (s)	Mean time (s)	Mean speedup	Mean efficiency	Best updates/s
1	0.177989	0.198924	1.00	1.000	4.486×10^9
2	0.088981	0.097045	2.05	1.025	8.973×10^9
4	0.053845	0.058491	3.40	0.850	1.483×10^{10}
8	0.034985	0.035216	5.65	0.706	2.282×10^{10}
12	0.037691	0.038856	5.12	0.427	2.118×10^{10}
14	0.035455	0.035608	5.59	0.399	2.252×10^{10}

The slightly superlinear mean efficiency at 2 threads is most likely caused by measurement noise and cache effects when comparing against the serial baseline.

The large case scales much better because there is more work per timestep. The best performance is reached around 8 threads, and 14 threads remains close to that result, but additional threads do not produce proportional speedup. This is consistent with a memory-bound stencil-like computation [10, 9].

9 Plots

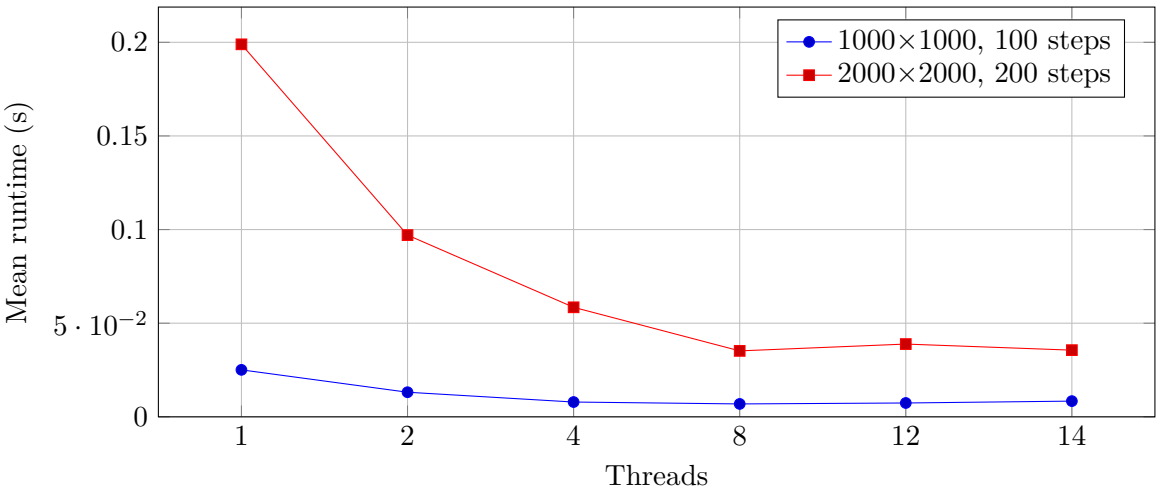

Figure 1: Mean runtime as a function of thread count.

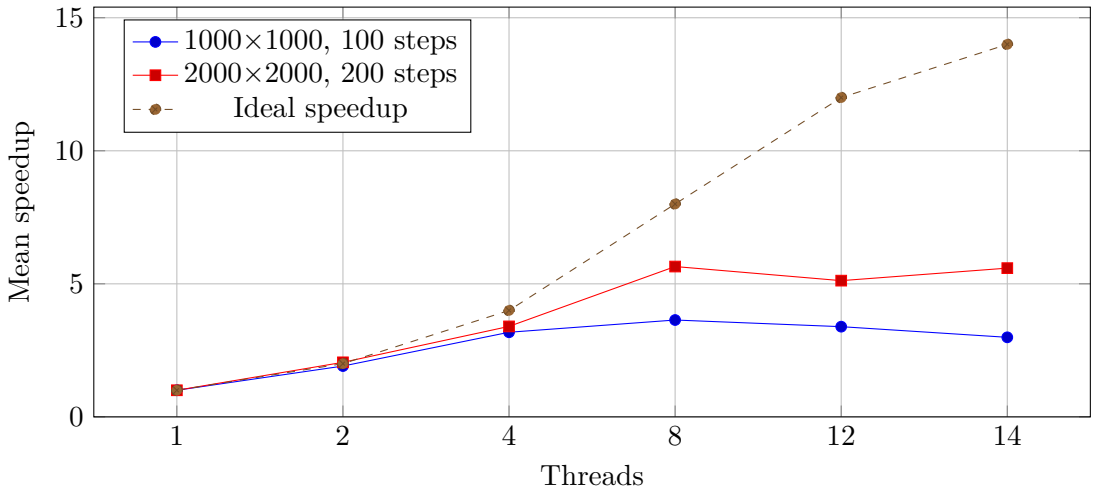

Figure 2: Mean speedup relative to the optimized serial baseline.

10 Discussion

The results support three main conclusions.

First, serial optimization mattered a lot. The earlier serial measurements showed an improvement of about 5× after removing bounds checks, inlining the neighbor computation, and reducing arithmetic overhead. This confirms that for stencil-like codes, constant-factor optimization in the hot loop is crucial.

Second, the parallelization strategy is effective. Static row-block partitioning is easy to implement and works well because each thread performs similar work on contiguous memory regions. Persistent workers further reduce overhead by avoiding repeated thread creation and destruction.

Third, scalability is ultimately limited by memory behavior and synchronization overhead. The Game of Life update performs relatively little arithmetic per cell compared with the amount of data movement. As thread count increases, the program eventually stops scaling linearly because more threads compete for memory bandwidth and the cost of synchronizing once per timestep becomes more visible.

For the medium case, the best mean runtime occurs at 8 threads, and using 12 or 14 threads is slightly worse. For the large case, scaling is still beneficial at 8 threads, and 14 threads remains close to the best result, but the gain is far below ideal linear speedup.

11 Conclusion

This project implemented a performant 2D Game of Life in C and parallelized it with Pthreads. The serial optimizations reduced the constant factor of the computation by around 5×, producing a strong baseline for parallelization. The final parallel implementation used persistent worker threads, static row-block decomposition, and a macOS-compatible synchronization design based on one mutex and condition variables.

The program achieved good scaling, especially on the larger problem size, where the mean speedup reached about 5.65× at 8 threads. Beyond that point, additional threads produced limited benefit, indicating that the computation is increasingly constrained by memory traffic and synchronization overhead rather than by lack of parallel work.

Overall, the project demonstrates that careful serial optimization is essential before parallelization, and that a simple, well-structured Pthreads design can provide strong performance while remaining understandable and portable.

References

[1] Martin Gardner. Mathematical Games—The Fantastic Combinations of John Conway's New Solitaire Game "Life". *Scientific American*, 223(4):120–123, 1970.

[2] Elwyn R. Berlekamp, John H. Conway, and Richard K. Guy. *Winning Ways for your Mathematical Plays, Volume 2.* Academic Press, 1982.

[3] David R. Butenhof. *Programming with POSIX Threads.* Addison-Wesley, 1997.

[4] IEEE and The Open Group. *IEEE Std 1003.1-2017, POSIX Programmer's Manual.* IEEE, 2018.

[5] Ulrich Drepper. *What Every Programmer Should Know About Memory.* Red Hat, 2007.

[6] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective.* Pearson, 3rd edition, 2016.

[7] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach.* Morgan Kaufmann, 6th edition, 2019.

[8] John D. McCalpin. Memory Bandwidth and Machine Balance in Current High Performance Computers. *IEEE Computer Society Technical Committee on Computer Architecture Newsletter*, 1995.

[9] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers.* CRC Press, 2010.

[10] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[11] The Clang Team. AddressSanitizer. LLVM/Clang Documentation.

[12] The Clang Team. UndefinedBehaviorSanitizer. LLVM/Clang Documentation.